

Curs 4

WordNet → funcționează ca o bază de date lexicografică a limbii engleze, în care cuvintele sunt grupate în seturi de sinonime numite *synset-uri*. Fiecare **synset** reprezintă un concept semantic distinct și conține relații logice între cuvinte, cum ar fi sinonime, antonime ETC. În contextul lucrului cu biblioteca NLTK, WordNet este folosit în special pentru lematizare, adică pentru transformarea cuvintelor la forma lor de bază corectă din dicționar (lemma). De exemplu, cuvântul *running* devine *run*, iar *better* devine *good*. Acest lucru este posibil deoarece WordNet înțelege categoria gramaticală și sensul lexical al cuvântului, ceea ce simplifică analiza semantică a textului. Fără de un algoritm simplu de *stemming* (care doar taie sufixe fără a verifica sensul), WordNet permite o reducere lingvistică corectă, păstrând sensul real al cuvintelor. Astfel, el este fundamental pentru aplicații precum analiza de sentiment, căutarea semantică, traducerea automată sau recunoașterea entităților, unde înțelegerea precisă a semnificației cuvintelor este esențială.

PorterStemmer este un algoritm folosit în procesarea limbajului natural pentru a reduce cuvintele la rădăcina lor de bază, eliminând sufixele comune. A fost creat de Martin Porter în anul 1980 și rămâne unul dintre cele mai utilizate instrumente pentru simplificarea cuvintelor în analiza textuală. Scopul său este de a uniformiza formele flexionate ale aceluiași cuvânt, astfel încât termeni precum *running*, *runs* și *ran* să fie tratați ca aceeași rădăcină – *run*. Prin această abordare, algoritmul contribuie la o mai bună organizare și comparare a textelor, deși uneori poate genera forme care nu sunt cuvinte reale.

PorterStemmer este conceput doar pentru limba engleză. El folosește reguli de tăiere a sufixelor *ing*, *-ed*, *-ly* etc., deci nu funcționează corect pentru română, unde avem alte forme flexionare (*-elor*, *-ile*, *-ului*, *-am*, *-ati*, etc.).

Se poate lucra **și fără PorterStemmer**, dar în acest caz fiecare formă a unui cuvânt va fi tratată ca un termen diferit, ceea ce poate duce la un vocabular mai mare și la o analiză mai puțin precisă. De exemplu, cuvintele *run*, *running* și *runs* vor fi considerate distincte, deși au același sens de bază. Fără stemming, rezultatele în căutare, clasificare sau analiză de text pot fi mai fragmentate, deoarece programul nu va recunoaște relația dintre formele flexionate. Totuși, în unele aplicații simple, precum afișarea sau numărarea cuvintelor, PorterStemmer poate fi omis fără a afecta semnificativ rezultatul.

Stemming (folosim SnowballStemmer pentru limba română)

```
stemmer_ro = snowballstemmer.stemmer('romanian')
stemmed_ro = stemmer_ro.stemWords(filtered_ro)
```

Procesul de *stemming* pentru limba română folosind biblioteca *snowballstemmer* → o extensie modernă a algoritmului Porter care suportă mai multe limbi.

`stemmer_ro = snowballstemmer.stemmer('romanian')`, se creează un obiect specializat pentru limba română, capabil să recunoască și să elimine sufixele specifice acestei limbi, precum *-elor*, *-ilor*, *-ului* sau *-am*. Acest pas permite algoritmului să reducă diferite forme flexionare ale aceluiași cuvânt la o rădăcină comună.

`stemmed_ro = stemmer_ro.stemWords(filtered_ro)`, aplică efectiv procesul de stemming asupra unei liste de cuvinte deja filtrate. Algoritmul parcurge fiecare element al listei și elimină terminațiile flexionare, returnând o nouă listă cu formele simplificate. De exemplu, cuvintele „fetele”, „frumoase”, „canta” și „cantecele” devin „fet”, „frumoas”, „cant” și „cantec”.

Rezultatul obținut nu conține întotdeauna cuvinte reale din punct de vedere gramatical, însă este foarte util în analiza statistică a textului, în clasificare sau în căutare semantică, unde interesează mai mult

structura de baza a cuvintelor decat forma lor corecta. Pentru aplicatii care necesita acuratete lingvistica, precum traducerea automata sau analiza semantica profunda, este recomandata lematizarea cu ajutorul modelelor lingvistice disponibile in biblioteca spaCy.

Ex 1

```
# !pip install stopwordsiso snowballstemmer
import nltk
nltk.download('punkt')
nltk.download('stopwords')
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import stopwordsiso as stopwordsiso
import snowballstemmer

text_en = """Each synset represents a distinct semantic concept and contains logical relations
between words,
such as synonyms and antonyms. In the context of working with the NLTK library, WordNet is mainly
used
for lemmatization, which means transforming words to their base dictionary form (lemma).
For example, the word running becomes run, and better becomes good. This is possible because
WordNet understands the grammatical category and lexical meaning of the word, simplifying
semantic analysis."""
print("TEXT IN ENGLEZA ")
print(text_en)
print()
# 1. Tokenizare (impartirea textului in cuvinte)
tokens_en = word_tokenize(text_en)
print("=== TOKENIZARE (EN) ===")
print(tokens_en)
print()
# 2. Elminare stopwords (cuvinte comune)
stop_words_en = set(stopwords.words('english'))
filtered_en = [word for word in tokens_en if word.lower() not in stop_words_en and word.isalpha()]
print("DUPA ELIMINAREA STOPWORDS (EN) ")
print(filtered_en)
print()
# 3. Stemming (reducerea cuvintelor la radacina)
stemmer_en = PorterStemmer()
stemmed_en = [stemmer_en.stem(word) for word in filtered_en]
print("STEMMING (EN) ")
print(stemmed_en)
print()
text_ro = """Fiecare synset reprezinta un concept semantic distinct si contine relatii logice intre
cuvinte, cum ar fi sinonime, antonime etc. In contextul lucrului cu biblioteca NLTK, WordNet este
folosit in special pentru lematizare, adica pentru transformarea cuvintelor la forma lor de baza corecta
din dictionar (lemma). De exemplu, cuvantul running devine run, iar better devine good. Acest lucru
este posibil deoarece WordNet intelege categoria gramaticala si sensul lexical al cuvantului, ceea ce
simplifica analiza semantica a textului."""
print("TEXT IN ROMANA ")
print(text_ro)
print()
# 1. Tokenizare (impartirea textului in cuvinte)
```

```

tokens_ro = word_tokenize(text_ro)
print("TOKENIZARE RO")
print(tokens_ro)
print()
# 2. Eliminare stopwords (folosim biblioteca stopwordsiso pentru limba romana)
stop_words_ro = stopwordsiso.stopwords("ro")
filtered_ro = [word for word in tokens_ro if word.lower() not in stop_words_ro and word.isalpha()]
print("DUPA ELIMINAREA STOPWORDS RO")
print(filtered_ro)
print()
# 3. Stemming (folosim SnowballStemmer pentru limba romana)
stemmer_ro = snowballstemmer.stemmer('romanian')
stemmed_ro = stemmer_ro.stemWords(filtered_ro)
print("STEMMING (RO) ")
print(stemmed_ro)
print()

```

spaCy ofera lematizare si POS tagging mult mai precise decat un stemmer simplu, pentru ambele limbi. Stopwords: sunt incluse in modelele spaCy; nu mai este nevoie de pachete externe pentru engleza/romana.

Biblioteca spaCy nu are o functie nativa de *stemming* (ea foloseste doar lematizare, care e mai precisa). Se poate folosi PorterStemmer din nltk pentru engleza si SnowballStemmer pentru romana. Vezi exemplu:

```

import spacy
from nltk.stem import PorterStemmer
import snowballstemmer

```

```

# Incarcam modelele pentru ambele limbi
nlp_en = spacy.load("en_core_web_sm")
nlp_ro = spacy.load("ro_core_news_sm")

```

```

# Stemmeri pentru fiecare limba
stemmer_en = PorterStemmer()
stemmer_ro = snowballstemmer.stemmer('romanian')
text_en = """Each synset represents a distinct semantic concept and contains logical relations between words,
such as synonyms and antonyms. In the context of working with the NLTK library, WordNet is mainly used
for lemmatization, which means transforming words to their base dictionary form (lemma).
For example, the word running becomes run, and better becomes good. This is possible because
WordNet understands the grammatical category and lexical meaning of the word, simplifying semantic analysis."""

```

```

print("TEXT IN ENGLEZA ")
print(text_en)
print()

```

```

# Procesam textul cu modelul pentru engleza
doc_en = nlp_en(text_en)

```

```

# 1. Tokenizare
tokens_en = [token.text for token in doc_en]
print(" TOKENIZARE (EN) ")
print(tokens_en)
print()

```

```

# 2. Eliminare stopwords si punctuatie
filtered_en = [token.text for token in doc_en if token.is_alpha and not token.is_stop]
print(" DUPA ELIMINAREA STOPWORDS (EN) ")
print(filtered_en)
print()

```

```
# 3. Lemmatizare
lemmas_en = [token.lemma_ for token in doc_en if token.is_alpha and not token.is_stop]
print(" LEMMATIZARE (EN) ")
print(lemmas_en)
print()
```

```
# 4. Stematizare (adaugat)
stems_en = [stemmer_en.stem(word) for word in filtered_en]
print(" STEMMING (EN) ")
print(stems_en)
print()
```

```
# 5. Etichetare gramaticala
print("PARTI DE VORBIRE (POS TAGGING - EN)")
for token in doc_en:
    if token.is_alpha:
        print(f"{token.text:<15} -> {token.pos_}")
print("\n" + "="*60 + "\n")
```

text_ro = """Fiecare synset reprezinta un concept semantic distinct si contine relatii logice intre cuvinte, cum ar fi sinonime, antonime etc. In contextul lucrului cu biblioteca NLTK, WordNet este folosit in special pentru lematizare, adica pentru transformarea cuvintelor la forma lor de baza corecta din dictionar (lemma). De exemplu, cuvantul running devine run, iar better devine good. Acest lucru este posibil deoarece WordNet intelege categoria gramaticala si sensul lexical al cuvantului, ceea ce simplifica analiza semantica a textului."""

```
print("TEXT IN ROMANA ")
print(text_ro)
print()
```

```
# Procesam textul cu modelul pentru romana
doc_ro = nlp_ro(text_ro)
```

```
# 1. Tokenizare
tokens_ro = [token.text for token in doc_ro]
print("TOKENIZARE (RO)")
print(tokens_ro)
print()
```

```
# 2. Eliminare stopwords si punctuatie
filtered_ro = [token.text for token in doc_ro if token.is_alpha and not token.is_stop]
print("DUPA ELIMINAREA STOPWORDS (RO)")
print(filtered_ro)
print()
```

```
# 3. Lemmatizare
lemmas_ro = [token.lemma_ for token in doc_ro if token.is_alpha and not token.is_stop]
print("LEMMATIZARE (RO)")
print(lemmas_ro)
print()
```

```
# 4. Stematizare (adaugat)
stems_ro = stemmer_ro.stemWords(filtered_ro)
print("STEMMING (RO)")
print(stems_ro)
print()
```

```
# 5. Etichetare gramaticala
print("PARTI DE VORBIRE (POS TAGGING - RO)")
for token in doc_ro:
    if token.is_alpha:
        print(f"{token.text:<15} -> {token.pos_}")
print()
```

WordNet este o baza de date lexicografica ce ajuta la lematizare corecta (adica gasirea formei de baza a unui cuvânt tinând cont de sensul sau gramatical). Fara WordNet nu putem face lematizare precisa, doar stemming (care taie sufixe fara a tine cont de sens) dar putem face tokenizare, eliminare stopwords si stemming fara nicio problema.

```
# Afisare parti de vorbire (doar tokeni alfabetici)
print(" ETICHETARE POS (toti tokenii alfabetici)|")
for tok, tag in pos_tags:
    if tok.isalpha():
        print(f"{tok:<15} -> {tag}")
print()
```

se parcurge o listă de perechi obținute prin funcția `pos_tag()`, unde fiecare element conține un cuvânt și eticheta sa gramaticală. Pentru fiecare pereche, se verifică dacă tokenul conține doar litere, folosind metoda `isalpha()`. Astfel, sunt excluse semnele de punctuație, cifrele sau alte simboluri care nu sunt relevante pentru analiza gramaticală.

Instrucțiunea `print(f"{tok:<15} -> {tag}")` are rolul de a afișa rezultatele într-un format ordonat și ușor de citit. F-string-ul (f"...") permite inserarea directă a valorilor variabilelor în text, iar expresia `{tok:<15}` indică faptul că fiecare cuvânt va fi afișat pe o lățime fixă de 15 caractere, aliniat la stânga. În acest fel, toate etichetele gramaticale apar frumos aliniate într-o coloană verticală, indiferent de lungimea cuvintelor.

Metode pentru similitudinea semantică a propoziției în NLP

Similitudinea semantică a propozițiilor în procesarea limbajului natural (NLP) implică determinarea modului în care două propoziții sau fraze sunt similare ca sens. Există diferite tipuri de măsuri de similitudine semantică care pot fi utilizate în NLP.

- Măsurile de similitudine bazate pe șiruri – permit compararea șirurilor de text în sine, și semnificația cuvintelor sau a propozițiilor. Exemple de măsuri de similitudine bazate pe șiruri: raportul de potrivire (**Ratio**) măsoară procentajul de caractere identice între două șiruri și este implementat în biblioteca `fuzzywuzzy` cu funcția `fuzz.ratio()`; **distanța de editare - Levenshtein**, **similitudinea Jaccard** etc.
- Măsurile de similitudine bazate pe modele lingvistice, utilizează modele de limbaj pre-antrenate pentru a codifica semnificația textului și pentru a compara reprezentările codificate pentru a măsura similitudinea. Exemple de măsuri de similitudine bazate pe modele lingvistice includ **similitudinea cosine**, BERT, Transformers și **rețelele Siamese** reprezintă un subtip specific de rețele neurale, adaptate pentru anumite scenarii de învățare a similitudinilor între perechi de date.
- Alte tipuri de măsuri de similitudine semantică includ măsuri bazate **pe corpus, cum ar fi analiza semantică latentă (LSA)** și măsuri de similitudine distributivă, care utilizează analiza statistică pentru a compara distribuția cuvintelor în corpusurile de text pentru a măsura similitudinea. Acestea au numeroase aplicații în motoarele de căutare bazate pe text, traducere automată, sisteme de răspuns la întrebări și multe altele.

Similitudinea propozițiilor are diverse aplicații în NLP. De exemplu, în recuperarea informațiilor, sarcina de a găsi documente relevante pentru o anumită interogare poate fi îmbunătățită prin măsurarea similitudinii dintre interogare și documente.

În clasificarea textului, similitudinea propozițiilor poate ajuta la identificarea categoriei unui text dat prin compararea acestuia cu alte texte din aceeași categorie. Mai mult, în rezumarea textului, similitudinea propozițiilor poate fi utilizată pentru a identifica propozițiile redundante și a le elimina din rezumat.

Similitudinea Jaccard este o tehnică de măsurare a asemănării dintre două seturi de date. În procesarea limbajului natural (NLP), similitudinea Jaccard este adesea folosită pentru a compara seturile de cuvinte în două propoziții și pentru a măsura suprapunerea dintre ele.

Similitudinea Jaccard este calculată ca dimensiunea intersecției a două seturi împărțită la dimensiunea uniunii celor două seturi. În cazul a două propoziții, seturile sunt create prin conversia fiecărei propoziții într-un set de cuvinte unice.

Tehnica de similitudine Jaccard poate fi utilizată în aplicații în care intrarea formată din texte scurte sau propoziții. De exemplu, poate fi folosit în analiza social media pentru a măsura asemănarea dintre două tweet-uri sau postări pe Facebook.

Formula este: $J(A,B) = |A \cap B| / |A \cup B|$

Similitudinea Jaccard este simplu de calculat și independentă de limbaj, ceea ce o face utilă pentru seturi mari de date.

Rezultatul similitudinii Jaccard este o valoare între **0 și 1** care reprezintă gradul de similitudine dintre cele două propoziții. O valoare de 1 indică faptul că cele două propoziții sunt identice, în timp ce o valoare de 0 indică faptul că cele două propoziții sunt complet diferite.

pip install python-Levenshtein

Levenshtein Distance, este o metrică care calculează numărul minim de operații necesare pentru a transforma un șir în altul. Operațiile pot fi inserări, ștergeri sau înlocuiri de caractere. Distanța de editare dintre două șiruri este numărul minim de operații necesare pentru a converti un șir în altul.

Edit Distance poate fi folosit pentru a găsi asemănări între două șiruri. Este utilizat în multe aplicații, cum ar fi verificarea ortografică, detectarea plagiatului și alinierea secvenței **ADN**.

distanța de editare dintre două șiruri X și Y **este definită ca numărul minim de operații necesare pentru a transforma X în Y.**

Operațiunile pot fi:

- Inserare:
- Ștergere
- Înlocuire

Calcularea distanței de editare implică crearea unei matrice în care fiecare celulă reprezintă numărul minim de operații necesare pentru a transforma un subșir X într-un subșir de Y. Algoritmul umple matricea luând în considerare toate operațiunile posibile și alegând-o pe cea cu numărul de operații minim.

Rezultatul algoritmului Edit Distance este numărul minim de operații necesare pentru a transforma X în Y.

În acest exemplu, definim o funcție levenshtein_distance care ia două propoziții ca intrare și calculează distanța Levenshtein dintre ele. Funcția returnează o valoare întreagă care reprezintă numărul minim de operații necesare pentru a transforma o propoziție în cealaltă.

Observatie:

Pentru a calcula similitudinea dintre două șiruri folosind distanța de editare, putem folosi formula:

$$\text{similitudine} = 1 - (\text{edit_distance}(X, Y) / \max(\text{len}(X), \text{len}(Y)))$$

Scorul de similitudine variază de la 0 la 1, unde 0 nu indică nicio similitudine și 1 indică o similitudine exactă.

Similitudinea cosinusului încorporărilor de cuvinte

Similitudinea cosinusului dintre doi vectori este o măsură a asemănării orientărilor lor. Acesta variază de la -1 la 1, unde 1 indică faptul că cei doi vectori sunt identici, 0 indică faptul că sunt ortogonali și -1 indică faptul că sunt diametral opuși. Similitudinea cosinusului dintre doi vectori de cuvinte măsoară similitudinea semnificațiilor lor, ținând cont de contextul în care apar.

Similitudinea cosinusului încorporărilor de cuvinte a fost utilizată pentru diverse sarcini NLP, cum ar fi clasificarea textului, analiza sentimentelor și recuperarea informațiilor. De exemplu, într-o sarcină de clasificare a textului, se calculează similitudinea cosinusului dintre propoziție și fiecare clasă, iar propoziția este atribuită clasei cu cel mai mare scor de similitudine. Într-o sarcină de analiză a sentimentelor, se calculează similitudinea cosinusului dintre propoziție și un set de cuvinte purtătoare de sentimente, iar sentimentul propoziției este determinat de polaritatea și magnitudinea scorurilor de similitudine.

Similitudinea cosinusului dintre doi vectori de cuvinte x și y poate fi calculată după cum urmează:

$$\text{cosine_similarity}(x, y) = (x \cdot y) / (\|x\| * \|y\|)$$

unde $(x \cdot y)$ este produsul scalar al lui x și y și $\|x\|$ și $\|y\|$ sunt normele Euclidiene lui x și y.

$$\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

Pentru a implementa similitudinea cosinusului încorporărilor de cuvinte, trebuie mai întâi să reprezentăm fiecare cuvânt din propoziție ca un vector într-un spațiu vectorial.

Acest lucru se poate face folosind încorporări de cuvinte pre-instruite, cum ar fi Word2Vec, GloVe sau fastText. Aceste încorporări de cuvinte pot fi obținute prin antrenarea unei rețele neuronale pe un corpus mare de text, cum ar fi Wikipedia sau un crawl web.

Aplicatie- similaritatea cosinus pentru cuvintele curs si seminar

Metoda 1

Folosind TF-IDF → Instalarea bibliotecilor necesare (dacă nu sunt deja instalate):

pip install pandas scikit-learn

Metoda 2 → Word2Vec → pip install gensim

<https://code.google.com/archive/p/word2vec/>

Funcția **fuzz.ratio** din biblioteca **fuzzywuzzy** măsoară gradul de asemănare între două șiruri de caractere și returnează un scor de similaritate în procente. Această funcție se bazează pe algoritmul de similaritate de raport al lui Levenshtein (distanța de editare).

Algoritmul Levenshtein măsoară numărul minim de operații (inserții, ștergeri, înlocuiri) necesare pentru a transforma un șir în altul.

Cu cât este mai mică distanța Levenshtein, cu atât șirurile sunt mai asemănătoare.

fuzz.ratio convertește distanța Levenshtein într-un scor de similaritate folosind următoarea formulă:

Similaritate = $[(\text{Ln min a sirurilor} - \text{Dist Levenshtein}) / \text{Ln min a sirurilor}] \times 100$

Scorul de similaritate este întotdeauna în intervalul [0,100] → 100 înseamnă șiruri identice, iar 0 înseamnă șiruri complet diferite.

Pentru șirurile "python" și "pyhton", distanța Levenshtein este 1 (o singură inserție).

Scorul de similaritate calculat de fuzz.ratio este $(6-1)/6 \times 100 \rightarrow 83,33$

pentru două șiruri: „curs” și „orar” distanța Levenshtein este 4.

Similaritate = $(4-4)/4 \times 100 = 0$, **scorul de similaritate** între „curs” și „orar” este **0**, indicând că cele două șiruri sunt complet diferite.

calculăm scorul de similaritate pentru șirurile „curs” și „temele”

Similaritate = $(4-6)/4 \times 100 = -50\%$ în implementarea funcției fuzz.ratio, scorul de similaritate ar fi ajustat la **0**, indicând că șirurile sunt foarte diferite.

Aplicatia 1

Calculează similaritatea cosinusului, similitudinea fuzz.ratio, similitudinea Jaccard și distanța Levenshtein între perechile de texte.

Biblioteci folosite

- **pandas**: utilizare fișiere CSV.

- **TfidfVectorizer (sklearn)**: transformă textele în vectori bazati pe frecvența termenilor și importanța lor în documente, util pentru calcularea similarității cosinus.
- **cosine_similarity (sklearn)**: calculează similaritatea cosinus între vectori, măsurând similaritatea de direcție între texte.
- **combinations (itertools)**: generează toate perechile posibile de texte din setul de date pentru a calcula similaritățile între ele.
- **fuzz (fuzzywuzzy)**: Furnizează metoda fuzz.ratio, care calculează similaritatea bazată pe distanța Levenshtein dintre două șiruri.
- **distance (Levenshtein)**: calculează distanța Levenshtein direct, adică numărul minim de operații necesare pentru a transforma un șir de caractere în altul.

Explicații:

sklearn.feature_extraction.text.TfidfVectorizer: este folosit pentru a transforma un set de texte într-o reprezentare vectorială bazată pe frecvența termenilor (TF-IDF). Termenii importanți sunt ponderați în funcție de frecvența lor în document și invers proporțional cu frecvența lor în întregul corpus (colecție de documente).

sklearn.metrics.pairwise.cosine_similarity: face parte din scikit-learn și calculează similaritatea cosinusului între vectorii sau matricele date.

Similaritatea cosinusului măsoară unghiul dintre doi vectori și furnizează un scor între -1 și 1. Scorul este 1 atunci când vectorii sunt identici și -1 atunci când sunt complet opuși.

itertools.combinations: este o funcție din biblioteca standard Python care furnizează toate combinațiile posibile de lungime specificată dintr-o secvență. A fost folosită pentru a obține toate perechile unice de texte din DataFrame pentru a calcula măsurile de similaritate între ele.

fuzzywuzzy.fuzz: este un modul din biblioteca fuzzywuzzy care furnizează funcții pentru a măsura asemănarea între șiruri de caractere. fuzz.ratio este o funcție care utilizează distanța de editare (Levenshtein) pentru a calcula un scor de similaritate între două șiruri. **Scorul este între 0 și 100, unde 100 înseamnă șiruri identice.** **Levenshtein.distance**: măsoară numărul minim de operații necesare pentru a transforma un șir în altul.

Distanța de mutare a cuvintelor

Word Mover's Distance (WMD) este o tehnică care măsoară similitudinea semantică dintre două propoziții prin calcularea distanței minime pe care cuvintele încorporate ale unei propoziții trebuie să o parcurgă pentru a ajunge la cuvintele încorporate ale celeilalte propoziții.

Distanța Word Mover poate fi utilizată în diverse activități NLP, cum ar fi gruparea documentelor, sugestia de interogare și rezumarea textului.

Logica matematică din spatele WMD implică calcularea distanței dintre cuvintele încorporate a două propoziții folosind încorporările Word2Vec. **Distanța dintre două cuvinte încorporate este calculată ca distanța euclidiană dintre vectorii lor.** Distanța totală dintre două propoziții este calculată ca distanța minimă pe care fiecare cuvânt încorporat într-o propoziție trebuie să o parcurgă pentru a ajunge la cel mai apropiat cuvânt încorporat în cealaltă propoziție.

Implementarea Word Mover's Distance implică următorii pași:

- Încărcăm încorporările Word2Vec pre-instruite.
- Tokenizăm propozițiile de intrare și convertim fiecare simbol la încorporarea Word2Vec corespunzătoare.
- Calculăm matricea distanței dintre cuvintele încorporate ale celor două propoziții.
- Determinăm distanța minimă pe care fiecare cuvânt încorporat într-o propoziție trebuie să o parcurgă pentru a ajunge la cel mai apropiat cuvânt încorporat în cealaltă propoziție.
- Calculăm distanța totală dintre cele două propoziții ca suma distanțelor minime împărțită la numărul total de cuvinte încorporate.

Pentru WMD, avem nevoie de preprocesarea cuvintelor pentru a le transforma în vectori de cuvinte încorporate, iar apoi vom utiliza acești vectori pentru a calcula distanța WMD. Vom folosi spațiul vectorial pre-încărcat Word2Vec pentru a obține încorporări semantice pentru cuvinte.

Pentru a implementa distanța Word Mover's se folosește biblioteca Gensim.

Atentie: descărcați un model Word2Vec pre-antrenat specificați calea corespunzătoare în variabila `word2vec_model_path`. Modelul Google News Word2Vec poate fi descărcat de la următoarea adresă URL: [Google Code Archive - Long-term storage for Google Code Project Hosting](https://code.google.com/archive/p/word2vec/).

<https://code.google.com/archive/p/word2vec/>

Problema rezolvata

```
#https://fasttext.cc/docs/en/crawl-vectors.html
from gensim.models import KeyedVectors
import os
# Specifica calea catre modelul FastText pentru romana
model_path = r"C:\Users\Mironela Pirnau\Downloads\cc.ro.300.bin"
# Verificam daca fisierul exista
if not os.path.exists(model_path):
    print(f" Eroare: fisierul model nu exista la calea: \n{model_path}")
else:
    print("Model romanesc gasit, se incarca...")
    # Incarcam modelul FastText
    model = KeyedVectors.load_word2vec_format(model_path, binary=True)
    print("Model incarcat cu succes!")
    print(f"Vocabular: {len(model.key_to_index):,} cuvinte\n")
    # Verificam cuvintele in vocabular
    word1, word2 = "curs", "seminar"
    if word1 in model.key_to_index and word2 in model.key_to_index:
        similarity = model.similarity(word1, word2)
        print(f"Similaritatea cosinus intre '{word1}' si '{word2}' este: {similarity:.4f}")
    else:
        print(f" Cel puțin unul dintre cuvinte ('{word1}', '{word2}') nu este in vocabularul modelului.")
```

Acest program are rolul de a încărca un model **FastText pre-antrenat pentru limba română** și de a calcula **similaritatea semantică** dintre două cuvinte (în exemplu, „curs” și „seminar”). El reprezintă

o primă etapă în înțelegerea modului în care pot fi folosite **încorporările de cuvinte (word embeddings)** în analiza limbajului natural (NLP).

Mai întâi, se importă biblioteca **Gensim**, care conține clasa **KeyedVectors** — folosită pentru a lucra cu modele **Word2Vec** și **FastText** pre-antrenate. Apoi se importă modulul **os**, necesar pentru a verifica dacă fișierul cu modelul există la calea indicată.

Se definește variabila **model_path**, care conține **calea completă către fișierul modelului FastText** descărcat anterior de pe site-ul oficial <https://fasttext.cc/docs/en/crawl-vectors.html>

Să se scrie un program Python care să calculeze **gradul de similaritate dintre toate perechile posibile de texte** din fișier, utilizând următoarele metode:

1. **Similaritatea cosinus** – calculată pe baza vectorilor TF-IDF obținuți din conținutul textelor;
2. **Similaritatea Fuzzy (fuzz ratio)** – care măsoară potrivirea aproximativă dintre două șiruri de caractere;
3. **Similaritatea Jaccard** – determinată în funcție de raportul dintre cuvintele comune și totalul cuvintelor unice din cele două texte;
4. **Distanța Levenshtein** – care indică numărul minim de operații de editare necesare pentru a transforma un text în celălalt.

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from itertools import combinations
from fuzzywuzzy import fuzz
from Levenshtein import distance # Import adăugat
import csv

file_path = 'c://2024//NLP/title.csv'
df = pd.read_csv(file_path)

if 'text' not in df.columns:
    print("Eroare în fișierul CSV.")
else:
    with open('c://2024//NLP/similarity_results.csv', 'w', newline='') as csv_file:
        csv_writer = csv.writer(csv_file)
        csv_writer.writerow(['text1', 'text2', 'similaritate_cosinus', 'similaritate_fuzz_ratio',
                              'similaritate_jaccard', 'distanța_levenshtein'])

    # Creează vectori TF-IDF pentru textele din dataframe
    vectorizer = TfidfVectorizer()
    vectors = vectorizer.fit_transform(df['text'])

    # Calculează similaritatea cosinusului, similitudinea fuzz.ratio, similitudinea Jaccard și distanța
    # Levenshtein între perechile de texte
    for pair in combinations(range(len(df['text'])), 2):
        text1, text2 = pair
        cosine_sim = cosine_similarity(vectors[text1], vectors[text2])[0][0]
        fuzz_ratio = fuzz.ratio(df['text'][text1], df['text'][text2])
        set1 = set(df['text'][text1].split())
        set2 = set(df['text'][text2].split())
```

```

jaccard_similarity = len(set1.intersection(set2)) / len(set1.union(set2))
levenshtein_distance = distance(df['text'][text1], df['text'][text2])
csv_writer.writerow([df['text'][text1], df['text'][text2], cosine_sim, fuzz_ratio,
jaccard_similarity, levenshtein_distance])
print("sf")

```

Sa se testeze aplicatia:

```

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from itertools import combinations
from fuzzywuzzy import fuzz
from Levenshtein import distance
from gensim.models import Word2Vec
from gensim.models import KeyedVectors
from nltk.tokenize import word_tokenize
import nltk
import csv

# Descărcați modelele Word2Vec pre-antrenate (de exemplu, GoogleNews-vectors-
negative300.bin)
# de la https://code.google.com/archive/p/word2vec/
word2vec_model_path = 'c://path_to_word2vec_model//GoogleNews-vectors-negative300.bin'
word2vec_model = KeyedVectors.load_word2vec_format(word2vec_model_path, binary=True)

nltk.download('punkt') # Descărcare de resurse pentru tokenizare

file_path = 'c://2024//NLP/title.csv'
df = pd.read_csv(file_path)

if 'text' not in df.columns:
    print("Eroare în fișierul CSV.")
else:
    with open('c://2024//NLP/similarity_results.csv', 'w', newline='') as csv_file:
        csv_writer = csv.writer(csv_file)
        csv_writer.writerow(['text1', 'text2', 'similaritate_cosinus', 'similaritate_fuzz_ratio',
'similaritate_jaccard', 'distanța_levenshtein', 'distanța_word_mover'])

    # Tokenizare pentru a obține liste de cuvinte
    df['tokenized_text'] = df['text'].apply(word_tokenize)

    # Calculează similaritatea cosinusului, similitudinea fuzz_ratio, similitudinea Jaccard,
    # distanța Levenshtein și distanța Word Mover între perechile de texte
    for pair in combinations(range(len(df['tokenized_text'])), 2):
        text1, text2 = pair
        cosine_sim = cosine_similarity(vectors[text1], vectors[text2])[0][0]
        fuzz_ratio = fuzz.ratio(df['text'][text1], df['text'][text2])

        set1 = set(df['text'][text1].split())

```

```

set2 = set(df['text'][text2].split())
jaccard_similarity = len(set1.intersection(set2)) / len(set1.union(set2))

levenshtein_distance = distance(df['text'][text1], df['text'][text2])

# Calculul distanței Word Mover
try:
    wmd_distance = word2vec_model.wmdistance(df['tokenized_text'][text1],
df['tokenized_text'][text2])
except Exception as e:
    print(f"Nu s-a putut calcula distanța Word Mover pentru perechea {text1}, {text2}.
Detaliu eroare: {e}")
    wmd_distance = None

csv_writer.writerow([df['text'][text1], df['text'][text2], cosine_sim, fuzz_ratio,
jaccard_similarity, levenshtein_distance, wmd_distance])

```

Rețele siameze

Rețelele siameze sunt rețele neuronale care sunt concepute pentru a învăța **similitudinea** dintre două intrări, de obicei propoziții sau imagini. Acestea constau din două subrețele identice care au aceleași ponderi și sunt instruite să producă reprezentări de caracteristici similare pentru intrări care sunt similare din punct de vedere semantic. Similitudinea dintre cele două intrări este apoi calculată folosind o metrică a distanței, cum ar fi distanța **euclidiană sau similitudinea cosinusului**.

Analiza semantică latentă (LSA)

Analiza semantică latentă (LSA) este o tehnică de procesare a limbajului natural (NLP) care utilizează o metodă matematică numită descompunere a valorii singulare (SVD) pentru a identifica modele latente în cadrul relațiilor dintre termeni și concepte într-un corpus de text. Această tehnică se bazează pe ipoteza distribuției, care presupune că cuvintele folosite în contexte similare **tind** să aibă semnificații similare.

LSA este utilizat pe scară largă în mai multe sarcini NLP, cum ar fi recuperarea informațiilor, clasificarea textului și rezumarea automată. De asemenea, poate fi utilizat pentru gruparea documentelor și sistemele de recomandare.

LSA folosind biblioteca **scikit-learn** a Python:

NLP si Vizualizare grafică a cuvintelor cheie dintr-un text

Norul de cuvinte (**Word Cloud**) este o vizualizare grafică a cuvintelor cheie dintr-un text, în care dimensiunea fiecărui cuvânt reflectă frecvența sa în textul original. Această tehnică simplifică și evidențiază cuvintele relevante, **oferind o imagine vizuală a distribuției lor în text**. În domeniul prelucrării limbajului natural (NLP), norul de cuvinte poate avea mai multe utilizări și beneficii:

Norul de cuvinte oferă o perspectivă **instantanee asupra frecvenței cuvintelor** într-un text. Cuvintele care apar mai des sunt reprezentate prin dimensiuni mai mari în nor.

Cuvintele cheie sau cele mai importante dintr-un text sunt evidențiate într-un nor de cuvinte. Aceasta oferă o privire rapidă asupra temelor sau conceptelor principale abordate.

În contextul analizei sentimentului, norul de cuvinte poate evidenția cuvintele cu încărcătură emoțională.

Etape: pip install **wordcloud**

- Citire date//curățare date, caractere speciale...
- Tokenizăm textul în cuvinte.///??
- Calculăm frecvența de apariție a fiecărui cuvânt.
- Definim un nor de cuvinte.
- Salvăm rezultatele într-un fișier CSV---frecventa cuvintelor///dictionar→

#norul de cuvinte pe baza de cuvinte lematizate →Rezolvare

```
from wordcloud import WordCloud
from collections import Counter
import matplotlib.pyplot as plt
import pandas as pd
import csv
import nltk
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

file_path = 'c://2024//NLP/title.txt'
with open(file_path, 'r', encoding='utf-8') as file:
    text_content = file.read()

words = text_content.split()
lemmatized_words = [lemmatizer.lemmatize(word.lower()) for word in words]
word_frequency = Counter(lemmatized_words)

wordcloud = WordCloud(width=400, height=400, background_color='white',
max_words=50).generate_from_frequencies(word_frequency)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()

csv_file_path = 'c://2024//NLP/frecventa_cuvinte.csv'
with open(csv_file_path, 'w', newline="", encoding='utf-8') as csv_file:
    csv_writer = csv.writer(csv_file)
    csv_writer.writerow(['Cuvant', 'Frecventa'])
    for word, frequency in word_frequency.most_common(50):
        csv_writer.writerow([word, frequency])

print(f"Norul de cuvinte a fost creat si salvat in '{csv_file_path}'.")
```